

VideoForge - Problems, Root Causes & Solutions

This document summarizes every issue encountered while running the VideoForge standalone generator (videoforge-backend\generate_video.py) and how each one was fixed. All code changes listed below are in videoforge-backend\generate_video.py.

Problem 1 - Playwright cannot launch Chromium (video rendering fails)

Symptom

```
STEP 5: Rendering scenes to video segments
Playwright recording failed: BrowserType.launch: Executable doesn't exist at
C:\Users\Asus\AppData\Local\ms-playwright\chromium_headless_shell-1234
\chrome-headless-shell-win64\chrome-headless-shell.exe
(Looks like Playwright was just installed or updated.)
Please run the following command to download new browsers:
    playwright install
```

Root cause

The installed Python package was playwright==1.62.0, which expects Chromium build 1234. The local browser cache (%LOCALAPPDATA%\ms-playwright) only contained builds 1217 and 1223. Audio generation (edge-tts) worked fine; only the HTML-to-video rendering step failed.

Solution

```
python -m playwright install chromium
```

Prevention

Pin the package version so the package/browser pairing cannot drift:

```
# videoforge-backend\pyproject.toml -> [project] dependencies
"playwright==1.62.0"
```

After a fresh install always run: python -m playwright install chromium.

Problem 2 - STEP 6 FFmpeg concat fails ("No such file or directory")

Symptom

```
FFmpeg concat error: Impossible to open
'videoforge-backend\projects\test1\renders\videoforge-backend/projects/test1/renders/sc_01.mp4'
```

Notice the doubled path prefix: the scene files that already had merged audio (sc_01.mp4, sc_02.mp4) could not be opened.

Root cause

The concat list (concat.txt) was written with relative paths:

```
concat_file.write_text("".join(f"file '{v.as_posix()}'\n" for v in video_clips), encoding="utf-8")
```

FFmpeg resolves paths inside a concat file relative to the concat file's own directory (not the current working directory). When the project was given as a relative path (--project-dir "videoforge-backend\projects\test1"), the paths were prefixed twice -> the files did not exist.

Fix - write absolute paths

```
# Absolute paths for every clip
entries = [v.resolve() for v in video_clips]
concat_file.write_text("".join(f"file '{v.as_posix()}'\n" for v in entries), encoding="utf-8")

# ...and pass the concat file itself by absolute path:
result = subprocess.run([
    "ffmpeg", "-y", "-f", "concat", "-safe", "0",
    "-i", str(concat_file.resolve()),
    "-c", "copy", "-movflags", "+faststart",
    str(output_path),
```

```
], capture_output=True, text=True, timeout=120)
```

(-safe 0 was already present, so C:/... paths are accepted.)

Problem 3 - Final video is "mute" (voice only ~12s, then silence)

Symptom

The final renders\video.mp4 had an audio stream, but silence dominated. Example: a 60-second video with voice only at 0-12s and 30-41s.

Root cause

Each scene's video clip was forced to its script template duration:

```
duration = max(template_duration, audio_duration) if audio_duration > 0 else template_duration
```

For a 30s scene with a 12.5s narration, $\max(30, 12.5) = 30\text{s}$ -> 17.5s of dead silence per scene. (Note: scenes\videos\scene-XX.mp4 are rendered silently with -an by design; audio is only attached in STEP 6.)

Fix - make narration drive the video length

```
duration = audio_duration + 1.0 if audio_duration > 0 else template_duration
```

Each scene is now cut to its narration length (+1s buffer) -> continuous voice, no mute gaps.

Problem 4 - Rendering takes ~50 minutes for an ~8-minute video

Symptom

Rendering a 6-scene script (timecodes up to 00:50:00) took 8-10 minutes per scene; not even scene 1 finished after 5 minutes.

Root cause

STEP 5 rendered each scene using the script timing:

```
duration = float(scene.get("estimated_duration") or scene["duration_seconds"])
```

Playwright records the page in real time for that duration, so scene timecodes of 8-10 minutes forced 8-10 minutes of recording - but the final video is only as long as the narration. This was pure wasted time.

Fix - render at narration length too

```
audio_duration = float(scene.get("audio_duration", 0))
duration = audio_duration + 1.0 if audio_duration > 0 else float(scene.get("estimated_duration") or scene["duration_seconds"])
```

Render time now equals total narration (+ small buffer) instead of the sum of the timecodes.

Problem 5 - Old audio reused after editing the script (video length unchanged)

Symptom

More narration text was pasted into script.txt, but the final video length did not change.

Root cause

STEP 4 caches the TTS files and never notices the script changed:

```
if output_path.exists() and output_path.stat().st_size > 1000:
    log.info(" Scene %d: audio exists, skipping", idx + 1)
```

If audio\narration\scene-01.mp3 already exists (>1000 bytes), the new text is never sent to the TTS engine.

Manual fix

```
Remove-Item "videoforge-backend\projects\test1\audio\narration\*.mp3"
# or the whole folder:
Remove-Item -Recurse "videoforge-backend\projects\test1\audio"
```

Then re-run the generator. Look for "requesting TTS" in the log - if you see "audio exists, skipping", the cache was

still in place.

Recommended (optional) enhancement

Make the cache hash-aware: store a sidecar scene-XX.txt containing md5(narration) next to the MP3 and only reuse audio when the hash matches.

Problem 6 - generate_video.py restored after accidental deletion

Symptom

```
python.exe: can't open file '...\generate_video.py': [Errno 2] No such file or directory
```

Root cause / fix

The script file was accidentally deleted together with output folders. It was recovered from the Windows Recycle Bin (the delete happened minutes earlier, so the file - including the applied fixes - was intact). The project folder projects\test1 was restored/recreated the same way.

Tip: make a backup copy of generate_video.py (or keep the repo under version control) so rapid fixes are never lost.

Key Notes

- Scene timecodes ([SCENE 1 ... - 0:00 - 8:00]) are now informational only. Video length = total narration + 1s buffer per scene. To make a longer video, add more narration text; editing the timecodes alone has no effect (except as a fallback when a scene has no audio).
- Which file to play: the voiced final video is renders\video.mp4 (identical to renders\final.mp4). The per-scene segments scenes\videos\scene-XX.mp4 are always silent (-an).
- ffmpeg / ffprobe are bundled in videoforge-backend\bin\ and are NOT on the system PATH - call them by full path.
- TTS fallback: if a scene's TTS fails, a silent placeholder is used and the scene falls back to its script duration_seconds.

Verifying the output

1) Streams - must show both 'video' and 'audio':

```
& "C:\Users\Asus\Downloads\video generation automation\videoforge-backend\bin\ffprobe.exe" -v error  
-show_entries stream=codec_type -of csv=p=0 "C:\Users\Asus\Downloads\video generation  
automation\videoforge-backend\projects\test1\renders\video.mp4"
```

2) Level - mean_volume should be clearly above silence (e.g. -25 dB):

```
& "C:\Users\Asus\Downloads\video generation automation\videoforge-backend\bin\ffmpeg.exe" -i  
"C:\Users\Asus\Downloads\video generation automation\videoforge-backend\projects\test1\renders\video.mp4" -map 0:a:0 -af volumedetect -f null  
NUL
```

3) Silence map - there should be no long silent stretches between scenes:

```
& "C:\Users\Asus\Downloads\video generation automation\videoforge-backend\bin\ffmpeg.exe" -i  
"C:\Users\Asus\Downloads\video generation automation\videoforge-backend\projects\test1\renders\video.mp4" -map 0:a:0 -af  
silencedetect=n=-40dB:d=0.5 -f null NUL
```

Syntax check for edited files:

```
python -c "import ast; ast.parse(open(r'videoforge-backend\generate_video.py',  
encoding='utf-8').read()); print('SYNTAX OK')"
```

File-change summary (videoforge-backend\generate_video.py)

Area	Before	After
Concat list (STEP 6)	relative v.as_posix()	v.resolve() (absolute) +
concat_file.resolve()		
Composition duration (STEP 6)	max(template, audio)	audio_duration + 1.0 when
audio exists		

```
Render duration (STEP 5)      | scene["duration_seconds"]      | audio_duration + 1.0 when  
audio exists
```

Optional pending tasks

- Pin playwright==1.62.0 in pyproject.toml.
- Add 'python -m playwright install chromium' to the README setup steps.
- Implement hash-aware TTS caching so script edits auto-invalidate old audio.
- Keep generate_video.py under version control to avoid future data loss.